UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

MATERIA: Fundamentos de Inteligencia Artificial

DOCENTE: Mtro. Javier García Lara

ALUMNA: Denise Estrada Franco

TEMA: 2.4 ALGORITMOS DE BÚSQUEDA CIEGA (BFS/DFS)

SEMANA 2: 19-25 octubre

GRUPO: 171

CARRERA: Tecnologías de la Información e Innovación Digital

MATRÍCULA: 24305019

FECHA DE ENTREGA: 25 de octubre de 2026

1. DATOS DEL PROBLEMA

Se desea encontrar un camino que lleve a un agente desde la entrada hasta la salida de un laberinto. El laberinto es una cuadrícula de n × n celdas; cada celda es libre o pared. El agente solo puede moverse a una celda libre vecina (arriba, derecha, abajo, izquierda) y cada movimiento cuesta 1. La entrada es la celda (1, 1) y la salida es la celda (n-2, n-2). Es un problema clásico de búsqueda sin información: el agente no sabe qué tan cerca está de la salida (Russell y Norvig, 2020; Bohm, 2021).

Datos de entrada utilizados

Dato	Valor
Tamaños de laberinto	11×11, 21×21, 31×31, 41×41 y 61×61
Tipos de laberinto	**Perfecto:** sin ciclos, un solo camino posible (el grafo es un árbol). **Con lazos:** se derriba 10 % de las paredes internas para crear ciclos y caminos alternativos.
Generación	Backtracker iterativo con semilla (reproducible). Semilla 42 para la tabla principal; semillas 1000 a 1029 (30 laberintos por configuración) para los promedios.
Orden de vecinos	Arriba, derecha, abajo, izquierda (fijo, para que cualquiera pueda repetir el resultado).
Métricas	Longitud del camino (pasos), nodos expandidos, tamaño máximo de la frontera y tiempo (mediana de 15 ejecuciones, en ms).
Software	Python 3, NetworkX (grafo y validación), Matplotlib (gráficas).

Ejemplo pequeño (9×9, semilla 7, con lazos). # = pared, . = libre:

```
#########
#.#.....#
#.###.#.#
#.....#.#
#####.#.#
#...#...#
#.#.###.#
#.......#
#########
```

Este ejemplo produce un grafo con 33 nodos y 34 aristas.

2. PLANTEAMIENTO DEL PROBLEMA

Problema: Dado un laberinto, encontrar una secuencia de movimientos que lleve de la entrada a la salida usando únicamente búsqueda ciega, y comparar el desempeño de BFS y DFS

Preguntas: 1. ¿Cuál algoritmo entrega el camino más corto?, 2. ¿Cuál expande menos nodos y es más rápido?, 3. ¿Cómo cambia el resultado cuando el laberinto tiene ciclos?

Hipótesis: BFS siempre devolverá un camino mínimo (costo uniforme igual a 1). En un laberinto perfecto ambos algoritmos encontrarán el mismo camino, porque solo existe uno. En un laberinto con lazos DFS expandirá menos nodos, pero su camino será más largo.

3. FORMULACIÓN COMO PROBLEMA DE BÚSQUEDA

Componente	Definición en este problema
Estado	Una celda libre (fila, columna) donde está el agente.
Estado inicial	(1, 1)
Acciones	Arriba, derecha, abajo, izquierda, solo si la celda destino es libre.
Modelo de transición	Resultados (s, a) = celda vecina en la dirección a
Prueba de meta	La prueba de meta consiste en verificar si el estado actual es (n-2, n-2)
Costo de camino	1 por movimiento; el costo de un camino es su número de pasos
Espacio de estados (grafo)	G = (V, E): V = celdas libres; E = pares de celdas libres adyacentes (no dirigido, peso 1). Es un laberinto perfecto |E|= |V| -1 (árbol); con lazos |E| > |V| - 1.

Los nodos y aristas de cada laberinto se muestran en la tabla 1 de la sección 6. La figura 1 dibuja el grafo del ejemplo de 9×9.

4. ALGORITMOS Y PROCEDIMIENTO

Ambos algoritmos mantienen una frontera de nodos por expandir y un registro de nodos visitados para no repetir estados. Solo cambia la estructura de la frontera (Russell y Norvig, 2020; Bohm, 2021).

BFS revisa primero las celdas más cercanas a la entrada: primero las que están a 1 paso, después a 2, luego a 3…, como una gota de agua que se expande. Por eso, cuando llega a la salida, el camino que encontró es el más corto. DFS se mete por un pasillo hasta el fondo y, si es un callejón sin salida, regresa y prueba otro; puede llegar rápido, pero un camino largo. En el código la diferencia es una sola línea: BFS saca el siguiente nodo del inicio de la lista (cola, popleft()) y DFS lo saca del final (pila, pop ()).

Propiedad	BFS (anchura)	DFS (profundidad)
Frontera	Cola FIFO (collections.deque)	Pila LIFO (lista de Python)
Orden de exploración	Por niveles : primero todos los nodos a 1 paso, luego a 2 pasos….	Se mete lo más hondo posible por una rama y retrocede al toparse con un callejón
Completo (grafo finito, con visitados)	Sí	Sí
Óptimo (costo uniforme = 1)	Sí, devuelve el camino con menos pasos	No, devuelve el primero que encuentra
Tiempo	O(V + E); en forma de árbol de búsqueda: O(b^(d+1))	O(V + E); en forma de árbol de búsqueda: O(b^m)
Espacio	O(V) con registro de visitados; en forma de árbol de búsqueda: O(b^(d+1))	O(V) con registro de visitados; en forma de árbol de búsqueda: O(b·m)

b= factor de ramificación, d= profundidad de la solución más superficial, m= profundidad máxima, V = nodos, E = aristas.

Procedimiento

1. Generar el laberinto (cuadrícula de 0 y 1) con una semilla
2. Convertirlo en un grafo de NetworkX y en una lista de adyacencia con orden de vecinos fijo.
3. Ejecutar BFS: sacar el nodo más antiguo de la cola, contarlo como expandido, terminar si es la meta; si no, encolar sus vecinos no vistos y guardar su padre.
4. Ejecutar DFS: sacar el nodo más reciente de la pila; si ya fue visitado, ignorarlo; si no, marcarlo, contarlo, terminar si es la meta y apilar sus vecinos (al revés, para respetar el orden de movimientos).

5. Reconstruir el camino siguiendo los padres desde la meta hasta la entrada y comprobar que cada paso es una arista válida.
6. Validar que la longitud de BFS coincide con nx.shortest_path_length y registrar pasos, nodos expandidos, frontera máxima y tiempo.
7. Repetir para cada tamaño y tipo de laberinto; promediar sobre 30 semillas; graficar.

5. RESULTADOS Y SOLUCIÓN

5.1 Diagrama del grafo

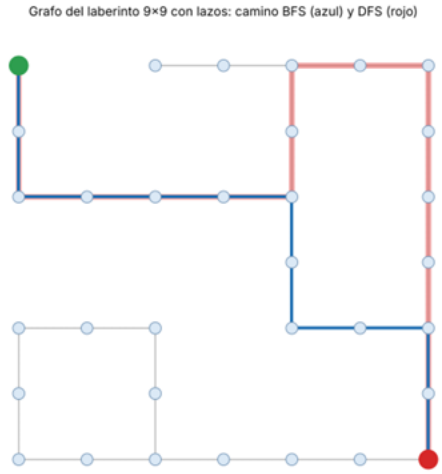


Figura 1. Grafo del laberinto de 9x9 (verde = entrada, rojo = salida), BFS (azul) usa 12 pasos y DFS (rojo) 16 pasos.

5.2 Solución sobre un laberinto de 21x21 con lazos (semilla 42)

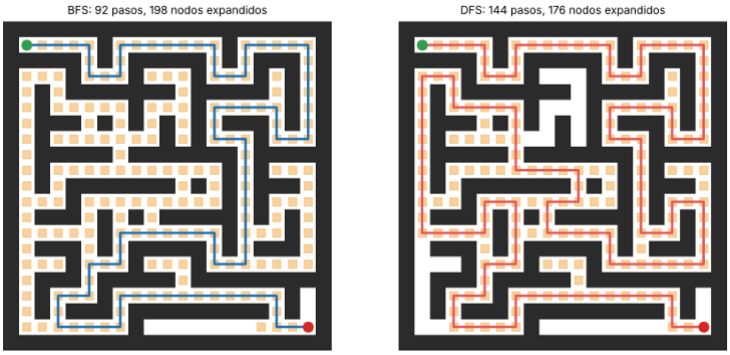


Figura 2. Camino encontrado (línea) y nodos expandidos (cuadro naranja) por BFS (izquierda) y DFS (derecha). Verde = entrada, rojo = salida.

SOLUCIÓN: En el laberinto de 21×21 con lazos, BFS encontró un camino de 92 pasos (el mínimo, confirmado con NetworkX) expandiendo 198 nodos. DFS encontró un camino de 144 pasos (52 pasos más largo) expandiendo 176 nodos. Ambos llegan a la salida; solo BFS garantiza el camino más corto.

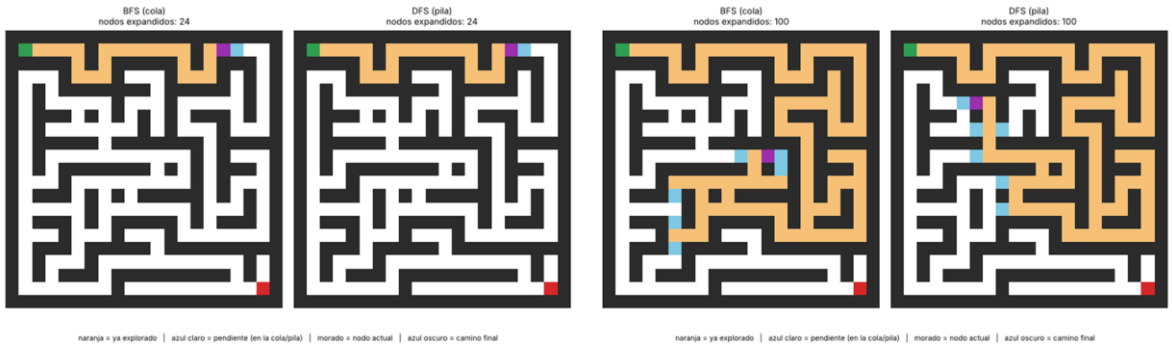


Figura 3. Dos momentos de la animación: a los 24 y a los 100 nodos expandidos. Naranja = explorado, azul claro = pendiente en la cola (BFS) o pila (DFS), morado = nodo actual. El resultado final es la figura 2.

5.3 Comparación (semilla 42)

Tabla 1. Resultados con la semilla 42. Exp = nodos expandidos; Front = tamaño máximo de la frontera; ms = tiempo mediano

Laberinto	Tamaño	Nodos	Aristas	Pasos BFS	Pasos DFS	Exp. BFS	Exp. DFS	Front. BFS	Front. DFS	ms BFS	ms DFS
Perfecto	11×11	49	48	24	24	25	25	1	1	0.008	0.011
Perfecto	21×21	199	198	132	132	190	181	4	4	0.056	0.096
Perfecto	31×31	449	448	240	240	398	275	6	5	0.162	0.141
Perfecto	41×41	799	798	376	376	671	661	10	12	0.331	0.379
Perfecto	61×61	1799	1798	264	264	307	289	4	4	0.096	0.151

Laberinto	Tamaño	Nodos	Aristas	Pasos BFS	Pasos DFS	Exp. BFS	Exp. DFS	Front. BFS	Front. DFS	Ms BFS	Ms DFS
Con lazos	11×11	50	50	24	24	25	25	1	1	0.008	0.011

Con lazos	21×21	207	214	92	144	198	176	11	12	0.060	0.079
Con lazos	31×31	468	486	100	228	435	255	11	18	0.154	0.111
Con lazos	41×41	835	870	84	108	596	170	14	16	0.186	0.097
Con lazos	61×61	1883	1966	164	252	1883	283	25	30	1.099	0.195

5.4 Promedios sobre 30 laberintos por configuración

Tabla 2. Promedios con las semillas 1000 a 1029.

Laberinto	Tamaño	Pasos BFS	Pasos DFS	Exp. BFS	Exp DFS	Ms BFS	Ms DFS
Perfecto	11×11	28.9	28.9	34.7	36.1	0.011	0.016
Perfecto	21×21	90.1	90.1	120.4	132.2	0.037	0.059
Perfecto	31×31	206.1	206.1	311.0	311.0	0.108	0.171
Perfecto	41×41	307.3	307.3	493.5	518.9	0.199	0.334
Perfecto	61×61	569.6	569.6	979.2	1047.3	0.457	0.744
Con lazos	11×11	24.3	26.8	37.7	33.4	0.011	0.015
Con lazos	21×21	52.9	75.7	144.6	108.8	0.042	0.050
Con lazos	31×31	82.0	140.3	417.6	209.5	0.149	0.106
Con lazos	41×41	108.1	228.7	756.7	364.1	0.298	0.202
Con lazos	61×61	164.1	360.4	1817.1	603.5	0.869	0.393

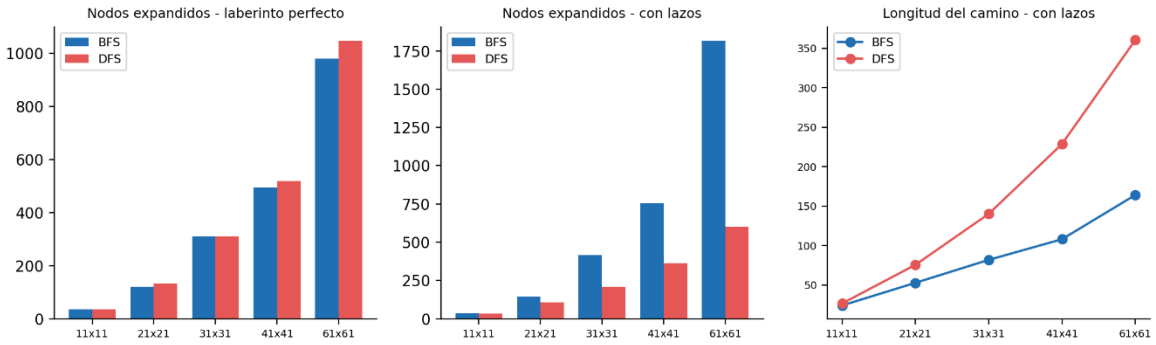


Figura 4. Promedios de nodos expandidos (izquierda y centro) y longitud del camino en laberintos con lazos (derecha).

6. ANÁLISIS DE COMPLEJIDAD

Complejidad teórica. En un grafo con V nodos y E aristas, BFS y DFS visitan cada nodo y cada arista a lo sumo una vez, así que el tiempo es O(V + E) en el peor caso. En el laberinto, cada celda tiene como máximo 4 vecinos y E es del orden de V, por lo que el costo crece casi linealmente con el número de celdas libres. En espacio, ambos guardan el conjunto de visitados (O(V)); la frontera de BFS crece con el ancho del área explorada y la de DFS con la profundidad (Russell y Norvig, 2020).

Lo observado.

- Calidad del camino. BFS fue óptimo en todas las corridas, validado mediante la comparación con nx.shortest_path_length. En los laberintos perfectos, BFS y DFS obtuvieron la misma longitud porque existe una única ruta entre la entrada y la salida; por ejemplo, ambos promediaron 569.6 pasos en laberintos de 61×61.
- Nodos expandidos. En los laberintos perfectos, BFS y DFS expandieron cantidades similares; para 61×61, los promedios fueron 979.2 y 1047.3, respectivamente. En laberintos con lazos, BFS expandió aproximadamente tres veces más nodos que DFS en 61×61 (1817.1 frente a 603.5), ya que BFS explora por niveles mientras DFS profundiza por una rama.
- Frontera. En todos los casos, la frontera máxima fue relativamente pequeña (hasta 25 nodos en BFS y 30 en DFS para el laberinto de 61×61 con lazos, tabla 1) porque los pasillos son angostos. La pila de DFS puede contener nodos repetidos, ya que se marcan como visitados al expandirlos.
- Tiempo. En los promedios de 30 laberintos, todos los tiempos registrados fueron inferiores a 1 ms en esta ejecución. En la tabla de una sola semilla, algunos tiempos pueden superar ese valor; las mediciones son sensibles al equipo y a la carga del sistema. En laberintos con lazos de 61×61, DFS promedió 0.393 ms frente a 0.869 ms de BFS.

Limitaciones. Los laberintos se generan aleatoriamente, por lo que los resultados de una sola semilla pueden variar y no necesariamente crecer de manera monótona con el tamaño. Los promedios de 30 semillas ayudan a reducir el efecto de casos particulares.

7. CONCLUSIONES

BFS es la opción correcta cuando importa la longitud del camino (rutas, juegos, movimientos mínimos): garantiza el camino más corto cuando todos los pasos cuestan lo mismo, a cambio de explorar más nodos cuando hay muchas rutas alternativas.

DFS es útil cuando basta con encontrar alguna solución, o cuando se quiere recorrer todo el grafo (conectividad, generación de laberintos). Es rápido y expande menos nodos con lazos, pero no hay garantía de que el camino sea bueno.

La idoneidad depende de la estructura del problema: en un árbol (laberinto perfecto) los dos son equivalentes en calidad; los ciclos son los que hacen que la elección importe.

Ambos son “ciegos”: no usan ninguna pista de dónde está la meta, por eso expanden muchas celdas que se alejan de la salida (figura 2).

Medir con varias semillas, validar contra una implementación de referencia (NetworkX) y reportar el número de nodos expandidos además del tiempo hizo que las conclusiones fueran más confiables que usar un solo laberinto.

8. REFERENCIAS

Bohm, R. A. (2021). The fundamentals of search algorithms. Nova Science.

García Serrano, A. (2016). Inteligencia artificial: Fundamentos, práctica y aplicaciones (2.ª ed.). RC Libros.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), Proceedings of the 7th Python in Science Conference (pp. 11-15).

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Python Software Foundation. (s. f.). collections — Container datatypes. Python 3 documentation. https://docs.python.org/3/library/collections.html

Russell, S., & Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4.ª ed.). Pearson

9. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Herramienta: Chat GPT

Prompt resumido: Implementa BFS y DFS en Python para resolver un laberinto, generar información sobre el camino solución, comparar el tiempo y los nodos expandidos, y analizar la complejidad temporal y espacial**.**

Uso: Diseño del experimento y programación de BFS y DFS

Verificación: Se ejecutó el código y se compararon los resultados obtenidos con las tablas y figuras incluidas en la documentación.